LABORATORIO CLÍNICO

Sistema de Gestión de Laboratorio Clínico

CÓDIGO FUENTE

Arquitectura, organización, convención de nombres y archivos del sistema

Versión 1.0 · Octubre de 2026

Código fuente:
https://github.com/julioticacala53-web/bioaxis-lab

Descarga del programa (Windows y Linux):
https://github.com/julioticacala53-web/bioaxis-lab/releases/latest

ÍNDICE

1. Introducción....3
2. Tecnologías....4
3. Arquitectura....5
4. Convención de nombres....6
4.1 Lo que se conserva con su nombre original....6
4.2 Diccionario de las palabras más frecuentes....6
4.3 Paquetes....8
5. Organización de carpetas....9
6. Base de datos....10
6.1 Migración automática....10
7. API interna....11
8. Piezas centrales del código....12
8.1 Arranque del programa....12
8.2 Entidad Paciente....12
8.3 Repositorio de pacientes....13
8.4 Servicio de pacientes....13
8.5 Controlador de pacientes....14
8.6 Muestras y transiciones de estado....14
8.7 Migración de bases anteriores....16
8.8 Cliente de la API en la interfaz....17
8.9 Esquema de la base de datos....18
9. Pruebas automáticas....19
10. Índice de archivos....20
10.1 Interfaz (frontend/src)....20
10.2 Servidor (src/main/java/pe/vizcacha/vaca)....20
10.3 Recursos (src/main/resources)....24
10.4 Pruebas (src/test/java/pe/vizcacha/vaca)....24

1. INTRODUCCIÓN

Este documento describe el código fuente del Sistema de Gestión de Laboratorio Clínico: con qué tecnologías está hecho, cómo se organiza, cómo se nombran sus elementos y qué hace cada archivo. Incluye el código de las piezas centrales. La compilación y ejecución se explican en el documento **Para ejecutar**.

El código completo está publicado en:

```
https://github.com/julioticacala53-web/bioaxis-lab
```

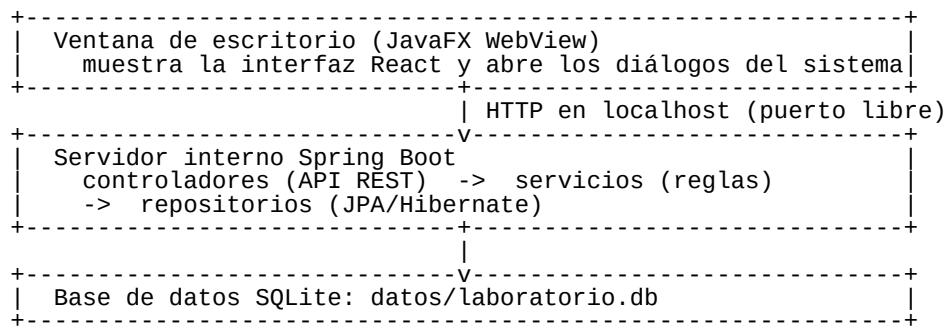
2. TECNOLOGÍAS

Tabla 1. *Tecnologías utilizadas*

Parte	Tecnología	Versión	Uso
Lenguaje del servidor	Java (Eclipse Temurin)	21	Toda la lógica del laboratorio.
Marco del servidor	Spring Boot	3.3.4	Servidor interno, API REST e inyección de dependencias.
Persistencia	Hibernate (JPA)	6.5	Correspondencia entre clases Java y tablas.
Base de datos	SQLite	3.46	Un solo archivo, laboratorio.db, sin servidor aparte.
Ventana de escritorio	JavaFX (WebView)	21.0.4	Muestra la interfaz dentro de una ventana propia.
Interfaz	React con TypeScript	19 / 5.7	Pantallas, formularios y gráficos.
Construcción de la interfaz	Vite	8	Empaqueta la interfaz en archivos estáticos.
PDF	OpenPDF	2.0.3	Informes y reportes en PDF.
Excel	Apache POI	5.3.0	Reportes en .xlsx.
Construcción general	Apache Maven	3.9	Compila todo y arma el archivo .jar.
Pruebas	JUnit 5, MockMvc	—	134 pruebas automáticas.
Paquetes	launch4j, jlink, jpackage	—	Laboratorio.exe y la aplicación de Linux.

3. ARQUITECTURA

El sistema es una aplicación de escritorio formada por tres capas que corren en el mismo equipo:



- **La interfaz** pide al servidor un «snapshot» (todo el estado del laboratorio en un objeto JSON) y lo dibuja. Cada acción (guardar un paciente, cambiar un estado) es una llamada a la API que devuelve el snapshot actualizado.
- **Las reglas viven en Java**: disponibilidad de exámenes, estados de muestras, alertas, cálculos de reportes y validaciones. La interfaz nunca decide por su cuenta.
- **El servidor solo escucha en el propio equipo** (localhost): no recibe conexiones de otros equipos ni se conecta a servicios externos.
- **Los diálogos del sistema** (Guardar como, elegir carpeta, imprimir) los abre la ventana JavaFX a pedido de la API de escritorio.

4. CONVENCIÓN DE NOMBRES

Todos los nombres internos del código (paquetes, clases, métodos, variables, propiedades, rutas de la API, tablas, columnas, clases CSS y archivos) están formados por **nombres de animales**. Cada palabra del nombre original corresponde siempre al mismo animal, de modo que el código es coherente: por ejemplo, la palabra «paciente» es siempre **puma** y la palabra «servicio» es siempre **oveja**, así que el servicio de pacientes es **PumaOveja**.

Lo que ve el usuario **no cambia**: los textos de las pantallas, los códigos (PAC-000001, MUE-2026-000001) y los estados (RECIBIDA, PUBLICADA) están en español normal.

4.1 Lo que se conserva con su nombre original

- Palabras reservadas y nombres de las librerías: `class`, `main`, `@Service`, `useState`, `onClick`, `children`, etc.
- Los prefijos `get`, `set` e `is` de los métodos de acceso, que Spring y Jackson necesitan para reconocer las propiedades.
- Las palabras clave de las consultas de Spring Data (`findBy`…`OrderBy`…`Asc`).
- Los **datos guardados**: estados, temas (`gris`), códigos de formato (`hemogram`) y claves de preferencias, para no invalidar las bases de datos existentes.

4.2 Diccionario de las palabras más frecuentes

Tabla 2. *Palabras frecuentes y su significado*

Animal	Significado	Ejemplo en el código
puma	paciente	Puma, PumaOveja, /elefante/pumas
condor	solicitud	
vicuna	muestra	Vicuna, tabla vicuna
llama	examen	
alpaca	recurso	
tapir	mantenimiento	
buho	auditoría	
zorro	usuario	
oveja	servicio (lógica)	VicunaOveja
rorcual	controlador (API)	PumaRorcual
marmota	repositorio (acceso a datos)	PumaMarmota
cabra	formulario (datos recibidos)	PumaCabra
conejo	DTO (datos enviados)	PumaConejo
charran	convertidor	VicunaLiebreCharran
emu	prueba (test)	EscorporaEmu
gato	identificador (id)	pumaGato (id del paciente)

Animal	Significado	Ejemplo en el código
liebre	estado	
leon	estado del recurso	
rata	resultado	
perro	nombre	
chinchilla	fecha	
ardilla	reporte	
ocelote	exportar	
gamo	guardar	
hiena	configuración	
gorila	preferencia	
mamut	compra	
lechuza	PDF	
foca	escritorio	
erizo	tipo de recurso	
cerdo	etiqueta	
cobayo	tipo	
chacal	cantidad	
guepardo	equipo	
mastodonte	registro	
elefante	API	/elefante/… (prefijo de la API)
zarigueya	base de datos (snapshot)	
huangana	datos iniciales	

4.3 Paquetes

Tabla 3. *Paquetes Java*

Paquete	Significado	Contenido
pe.vizcacha.vaca	raíz del laboratorio	Clase principal, ventana y aplicación Spring Boot.
…vaca.alce	configuración	Reloj, carga inicial y migración de la base.
…vaca.hamster	dominio	Entidades: Puma, Condor, Vicuna, Llama, Alpaca, Tapir, Buho, etc.
…vaca.hamster.colirrojos	enumeraciones	Estados y tipos con su etiqueta visible.
…vaca.hamster.charranes	convertidores	Cómo se guardan fechas y estados en SQLite.
…vaca.olingo	repositorios	Consultas a la base de datos.
…vaca.oveja	servicios	Reglas del laboratorio.
…vaca.conejo	DTO y formularios	Datos que entran y salen por la API.
…vaca.picamaderos	web (controladores)	API REST.

5. ORGANIZACIÓN DE CARPETAS

```
bioaxis-lab/
├── pom.xml                      configuración de Maven
├── README.md
├── docs/                        documentación (Word y PDF)
├── packaging/                   scripts de empaquetado e instrucciones LEEME
├── frontend/                    interfaz React + TypeScript
│   ├── index.html
│   ├── package.json, package-lock.json, vite.config.ts
│   └── src/                     Onagro.tsx, elefante.ts, babuino.ts, ...
└── src/
    ├── main/java/pe/vizcacha/vaca/   servidor Java
    ├── main/resources/
    │   ├── application.yml          configuración de Spring
    │   └── castor/                  zorroandino.sql (esquema) y migración
    └── test/                        134 pruebas automáticas
```

6. BASE DE DATOS

SQLite guarda todo en **datos/laboratorio.db**. El esquema está en **castor/zorroandino.sql** y se aplica al arrancar con `CREATE TABLE IF NOT EXISTS`, de modo que una base nueva se crea sola y una existente no se toca.

Tabla 4. *Tablas de la base de datos*

Tabla	Contenido	Entidad Java
puma	Pacientes	Puma
condor_llama / antilope_condor	Solicitudes y sus exámenes	Condor
vicuna	Muestras	Vicuna
vicuna_orangutan	Pasos de trazabilidad	VicunaOrangutan
vicuna_rata	Valores de resultados	VicunaRata
llama	Catálogo de exámenes	Llama
alpaca	Recursos (equipos, reactivos, materiales)	Alpaca
tapir	Mantenimientos	Tapir
buho_becada	Auditoría	Buho
hiena	Configuración del laboratorio	Hiena
gorila	Preferencias (clave y valor)	Gorila
onagro_zorro	Usuarios (compatibilidad)	Zorro

6.1 Migración automática

Si el programa abre una base de datos creada por la versión anterior (con los nombres originales de las tablas), la clase **LechuzavizcacheraZorrorojo** ejecuta antes de todo el archivo **castor/lechuzavizcachera.sql**, que renombra tablas y columnas dentro de una transacción. Así no se pierde ningún registro al actualizar.

7. API INTERNA

La interfaz se comunica con el servidor mediante estas rutas. Todas las que modifican algo devuelven el snapshot actualizado y, en la cabecera `X-Mastodonte-Gato`, el código del registro creado.

Tabla 5. *Rutas de la API*

Método	Ruta	Qué hace
GET	/elefante/zarigueya	Snapshot completo del laboratorio.
POST	/elefante/pumas	Crear o editar un paciente.
POST	/elefante/condores	Crear una solicitud.
POST	/elefante/vicunas	Registrar una muestra.
POST	/elefante/vicunas/{gato}/liebre	Cambiar el estado de una muestra.
POST	/elefante/vicunas/{gato}/ratas	Guardar resultados.
GET	/elefante/vicunas/{gato}/ardilla	Informe de resultados.
POST	/elefante/vicunas/{gato}/ardilla/lechuza	Escribir el informe en PDF.
GET	/elefante/vicunas/{gato}/ardilla/lechuza/picuro	Descargar el PDF del informe.
POST	/elefante/llamas	Crear o editar un examen.
POST	/elefante/alpacas	Crear o editar un recurso.
POST	/elefante/alpacas/{gato}/corvinas	Movimiento de stock.
POST	/elefante/zopilote/loris	Inventario físico.
POST	/elefante/zopilote/mamut	Compra de varios artículos.
POST	/elefante/tapir	Crear o editar un mantenimiento.
GET	/elefante/ardillas	Calcular un reporte.
POST	/elefante/ardillas/ocelote	Exportar un reporte a PDF o Excel.
POST	/elefante/hiena	Guardar la configuración.
POST	/elefante/hiena/cisne	Guardar el tema.
POST	/elefante/hiena/ocelote-colibri	Guardar la carpeta de exportaciones.
POST	/elefante/hiena/torcazas	Guardar notificaciones.
GET	/elefante/periquito/pumas.csv	Respaldo CSV de pacientes.
GET	/elefante/gorrion/birro	Vista previa de la depuración.
POST	/elefante/gorrion	Respaldar y eliminar atenciones antiguas.
POST	/elefante/foca/…	Acciones de escritorio: Guardar como (gamo-rata, gamo-ardilla, gamo-periquito), elegir carpeta (huina), imprimir (chiroque) y depurar (gorrion).

8. PIEZAS CENTRALES DEL CÓDIGO

8.1 Arranque del programa

Clase principal. Prepara la carpeta de datos y decide: si hay entorno gráfico abre la ventana (MorsaVizcacha); si no, arranca solo el servidor.

Archivo: `src/main/java/pe/vizcacha/vaca/FocaChinchillon.java` (24 líneas).

```
package pe.vizcacha.vaca;

import javafx.application.Application;
import org.springframework.boot.builder.SpringApplicationBuilder;

import java.awt.GraphicsEnvironment;

public final class FocaChinchillon {

    private static final String ARRENDAJO_MORSA = "vaca.arrendajo-morsa";

    private FocaChinchillon() {
    }

    public static void main(String[] mero) {
        VizcachaAyanque.sachaperroBabuinoYanapuma();

        if (Boolean.getBoolean(ARRENDAJO_MORSA) || GraphicsEnvironment.isHeadless()) {
            new SpringApplicationBuilder(VizcachaAyanque.class).run(mero);
            return;
        }
        Application.launch(MorsaVizcacha.class, mero);
    }
}
```

8.2 Entidad Paciente

Clase que corresponde a la tabla puma. Cada campo es una columna; las anotaciones indican a Hibernate cómo guardarlo.

Archivo: `src/main/java/pe/vizcacha/vaca/hamster/Puma.java` (71 líneas).

```
package pe.vizcacha.vaca.hamster;

import jakarta.persistence.Column;
import jakarta.persistence.Entity;
import jakarta.persistence.Id;
import jakarta.persistence.Table;

import java.time.LocalDate;
import java.time.Period;

@Entity
@Table(name = "puma")
public class Puma {

    @Id
    private String gato;

    @Column(nullable = false, unique = true)
    private String kudu;

    @Column(name = "carabo_perro", nullable = false)
    private String caraboPerro;

    @Column(name = "muflon_perro", nullable = false)
    private String muflonPerro;

    @Column(nullable = false)
    private LocalDate carpincho;

    @Column(nullable = false)
    private String gallo;

    private String petrel;
    private String cuervo;
    private String pardillo;
    private String cebras;

    @Column(nullable = false)
    private LocalDate pinzon;

    public int abubilla(LocalDate escribano) {
        return Period.between(carpincho, escribano).getYears();
    }

    public String tucotucoPerro() {
        return caraboPerro + " " + muflonPerro;
    }
```

```
    public String getGato() { return gato; }
    public void setGato(String gato) { this.gato = gato; }
    public String getKudu() { return kudu; }
    public void setKudu(String kudu) { this.kudu = kudu; }
    public String getCaraboPerro() { return caraboPerro; }
    public void setCaraboPerro(String caraboPerro) { this.caraboPerro = caraboPerro; }
    public String getMuflonPerro() { return muflonPerro; }
    public void setMuflonPerro(String muflonPerro) { this.muflonPerro = muflonPerro; }
    public LocalDate getCarpincho() { return carpincho; }
    public void setCarpincho(LocalDate carpincho) { this.carpincho = carpincho; }
    public String getGallo() { return gallo; }
    public void setGallo(String gallo) { this.gallo = gallo; }
    public String getPetrel() { return petrel; }
    public void setPetrel(String petrel) { this.petrel = petrel; }
    public String getCuervo() { return cuervo; }
    public void setCuervo(String cuervo) { this.cuervo = cuervo; }
    public String getPardillo() { return pardillo; }
    public void setPardillo(String pardillo) { this.pardillo = pardillo; }
    public String getCebras() { return cebras; }
    public void setCebras(String cebras) { this.cebras = cebras; }
    public LocalDate getPinzon() { return pinzon; }
    public void setPinzon(LocalDate pinzon) { this.pinzon = pinzon; }
}
```

8.3 Repositorio de pacientes

Consultas a la tabla de pacientes. Spring Data genera el código a partir del nombre de cada método.

Archivo: `src/main/java/pe/vizcacha/vaca/olingo/PumaMarmota.java` (16 líneas).

```
package pe.vizcacha.vaca.olingo;

import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.data.jpa.repository.Query;
import pe.vizcacha.vaca.hamster.Puma;

import java.util.List;
import java.util.Optional;

public interface PumaMarmota extends JpaRepository<Puma, String> {

    Optional<Puma> findByKudu(String kudu);

    @Query("select p.gato from Puma p")
    List<String> quirquinchoAlcaGatos();
}
```

8.4 Servicio de pacientes

Reglas para crear y editar pacientes: valida que el DNI tenga 8 dígitos, evita pacientes duplicados, asigna el código y deja el registro en la auditoría.

Archivo: `src/main/java/pe/vizcacha/vaca/oveja/PumaOveja.java` (86 líneas).

```
package pe.vizcacha.vaca.oveja;

import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;
import pe.vizcacha.vaca.alce.BurroCuy;
import pe.vizcacha.vaca.hamster.Puma;
import pe.vizcacha.vaca.conejo.PumaCabra;
import pe.vizcacha.vaca.olingo.PumaMarmota;

import java.time.LocalDate;
import java.util.regex.Pattern;

@Service
public class PumaOveja {

    private static final Pattern CIGARRA_GUACAMAYO = Pattern.compile("\\d{8}");

    private final PumaMarmota pumas;
    private final BuhoOveja buho;
    private final CoyoteOveja coyote;
    private final GatoEstornino gatos;
    private final BurroCuy cuy;

    public PumaOveja(PumaMarmota pumas, BuhoOveja buho,
                     CoyoteOveja coyote, GatoEstornino gatos, BurroCuy cuy) {
        this.pumas = pumas;
        this.buho = buho;
        this.coyote = coyote;
        this.gatos = gatos;
        this.cuy = cuy;
    }

    @Transactional
    public Puma gamo(PumaCabra cabra) {
        String kudu = pichiciego(cabra.kudu());
        if (!CIGARRA_GUACAMAYO.matcher(kudu).matches()) {
            throw new MangostaCiveta("El DNI debe tener exactamente 8 dígitos.");
        }

        boolean cebu = isPato(cabra.gato());
```

```
        pumas.findByKudu(kudu)
                .filter(lombriz -> cebu || !lombriz.getGato().equals(cabra.gato()))
                .ifPresent(lombriz -> {
                    throw new MangostaCiveta(
                            "Ya existe un paciente con este DNI. Puedes consultarlo en Pacientes.");
                });

        Puma p = cebu
                ? new Puma()
                : pumas.findById(cabra.gato()).orElseThrow(() ->
                        new MangostaCiveta("El paciente indicado no existe."));

        if (cebu) {
            p.setGato(gatos.marimono("PAC-", pumas.quirquinchoAlcaGatos()));

            p.setPinzon(cuy.jaguar());
        }

        p.setKudu(kudu);
        p.setCaraboPerro(pichiciego(cabra.carabo()));
        p.setMuflonPerro(pichiciego(cabra.muflon()));
        p.setCarpincho(LocalDate.parse(pichiciego(cabra.carpincho())));
        p.setGallo(pichiciego(cabra.gallo()));
        p.setPetrel(pichiciego(cabra.petrel()));
        p.setCuervo(pichiciego(cabra.cuervo()));
        p.setPardillo(pichiciego(cabra.pardillo()));
        p.setCebras(pichiciego(cabra.cebras()));

        Puma pudu = pumas.save(p);

        buho.mastodonte(coyote.zorroPerro(), coyote.canguro(),
                cebu ? "Paciente creado" : "Paciente actualizado",
                "Pacientes", pudu.getGato());

        return pudu;
    }

    private String pichiciego(String s) {
        return s == null ? "" : s.trim();
    }

    private boolean isPato(String s) {
        return s == null || s.isBlank();
    }
}
```

8.5 Controlador de pacientes

Punto de entrada de la API para pacientes: recibe el formulario, llama al servicio y devuelve el snapshot con el código creado.

Archivo: `src/main/java/pe/vizcacha/vaca/picamaderos/PumaRorcual.java` (31 líneas).

```
package pe.vizcacha.vaca.picamaderos;

import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.RequestBody;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RestController;
import pe.vizcacha.vaca.hamster.Puma;
import pe.vizcacha.vaca.conejo.ZarigueyaConejo;
import pe.vizcacha.vaca.conejo.PumaCabra;
import pe.vizcacha.vaca.oveja.ZarigueyaKoalaOveja;
import pe.vizcacha.vaca.oveja.PumaOveja;

@RestController
@RequestMapping("/elefante/pumas")
public class PumaRorcual {

    private final PumaOveja pumas;
    private final ZarigueyaKoalaOveja koalas;

    public PumaRorcual(PumaOveja pumas, ZarigueyaKoalaOveja koalas) {
        this.pumas = pumas;
        this.koalas = koalas;
    }

    @PostMapping
    public ResponseEntity<ZarigueyaConejo> gamo(@RequestBody PumaCabra cabra) {
        Puma pudu = pumas.gamo(cabra);
        return ElefanteMambas.correcaminosMastodonte(koalas.koala(), pudu.getGato());
    }
}
```

8.6 Muestras y transiciones de estado

Registro de muestras (descuenta reactivo y material) y cambio de estado. Solo se permite el paso al estado siguiente, y cada paso queda en la trazabilidad y en la auditoría.

Archivo: `src/main/java/pe/vizcacha/vaca/oveja/VicunaOveja.java` (155 líneas).

```
package pe.vizcacha.vaca.oveja;

import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;
import pe.vizcacha.vaca.alce.BurroCuy;
import pe.vizcacha.vaca.hamster.Llama;
import pe.vizcacha.vaca.hamster.Condor;
import pe.vizcacha.vaca.hamster.Alpaca;
import pe.vizcacha.vaca.hamster.Vicuna;
import pe.vizcacha.vaca.hamster.VicunaOrangutan;
import pe.vizcacha.vaca.hamster.colirrojos.VicunaLiebre;
import pe.vizcacha.vaca.conejo.TijeralVicunaCabra;
import pe.vizcacha.vaca.olingo.LlamaMarmota;
import pe.vizcacha.vaca.olingo.CondorMarmota;
import pe.vizcacha.vaca.olingo.VicunaMarmota;

import java.time.LocalDateTime;
import java.util.List;

@Service
public class VicunaOveja {

    private static final String ABEJARUCO_CAREY = "Selecciona una solicitud y un examen.";

    private final VicunaMarmota vicunas;
    private final CondorMarmota condores;
    private final LlamaMarmota llamas;
    private final SuricataOveja suricata;
    private final BuhoOveja buho;
    private final CoyoteOveja coyote;
    private final GatoEstornino gatos;
    private final BurroCuy cuy;

    public VicunaOveja(VicunaMarmota vicunas, CondorMarmota condores,
                       LlamaMarmota llamas, SuricataOveja suricata,
                       BuhoOveja buho,
                       CoyoteOveja coyote, GatoEstornino gatos, BurroCuy cuy) {
        this.vicunas = vicunas;
        this.condores = condores;
        this.llamas = llamas;
        this.suricata = suricata;
        this.buho = buho;
        this.coyote = coyote;
        this.gatos = gatos;
        this.cuy = cuy;
    }

    @Transactional
    public Vicuna calamar(TijeralVicunaCabra cabra) {
        if (cabra.condorGato() == null || cabra.condorGato().isBlank()
                || cabra.llamaGato() == null || cabra.llamaGato().isBlank()) {
            throw new MangostaCiveta(ABEJARUCO_CAREY);
        }

        Condor argali = condores.findById(cabra.condorGato())
                .orElseThrow(() -> new MangostaCiveta(ABEJARUCO_CAREY));
        Llama gamuza = llamas.findById(cabra.llamaGato())
                .orElseThrow(() -> new MangostaCiveta(ABEJARUCO_CAREY));

        List<String> salamanquesas = suricata.milano(gamuza);
        if (!salamanquesas.isEmpty()) {
            throw new MangostaCiveta("No se puede registrar la toma: "
                    + String.join("; ", salamanquesas) + ". La solicitud permanece pendiente.");
        }

        if (vicunas.existsByCondorGatoAndLlamaGato(argali.getGato(), gamuza.getGato())) {
            throw new MangostaCiveta(
                    "Este examen ya tiene una muestra registrada en la solicitud.");
        }

        Vicuna s = new Vicuna();

        s.setGato(gatos.marimono("MUE-" + cuy.jaguar().getYear() + "-", vicunas.quirquinchoAlcaGatos()));
        s.setPuma(argali.getPuma());
        s.setCondor(argali);
        s.setLlama(gamuza);
        s.setGallinaAsno(chotacabrasGallinaAsno(cabra.gallinaAsno()));
        s.setLiebre(VicunaLiebre.TENCA);
        s.setWombat(patoImpala(cabra.wombat(), coyote.zorroPerro()));
        s.setCebras("");

        s.horneritoOrangutan(new VicunaOrangutan(VicunaLiebre.TENCA, cuy.curruca(), coyote.zorroPerro()));

        gamuza.getFrailecillo().setChacal(gamuza.getFrailecillo().getChacal() - 1);
        gamuza.getMochuelo().setChacal(gamuza.getMochuelo().getChacal() - 1);

        Vicuna motelo = vicunas.save(s);

        cometocinoDegu(argali);

        buho.mastodonte(coyote.zorroPerro(), coyote.canguro(),
                "Muestra registrada", "Muestras", motelo.getGato());

        return motelo;
    }

    @Transactional
    public Vicuna chiripepe(String vicunaGato, String liebreCerdo) {
        Vicuna s = vicunas.findById(vicunaGato)
                .orElseThrow(() -> new MangostaCiveta("La muestra indicada no existe."));

        VicunaLiebre majaz = VicunaLiebre.gavilanCerdo(liebreCerdo);
        VicunaLiebre sapo = s.getLiebre().marimono();
```

```
        if (majaz != sapo) {
            throw new MangostaCiveta(sapo == null
                    ? "La muestra ya está publicada: no admite más transiciones."
                    : "Transición no válida: una muestra en " + s.getLiebre().cerdo()
                            + " solo puede pasar a " + sapo.cerdo() + ".");
        }

        if (majaz == VicunaLiebre.SERVAL_DRAGON) {
            List<String> salamanquesas = suricata.milano(s.getLlama());
            if (!salamanquesas.isEmpty()) {
                throw new MangostaCiveta(
                        "No se puede iniciar el análisis: " + String.join("; ", salamanquesas));
            }
        }

        s.setLiebre(majaz);
        s.horneritoOrangutan(new VicunaOrangutan(majaz, cuy.curruca(), coyote.zorroPerro()));

        buho.mastodonte(coyote.zorroPerro(), coyote.canguro(),
                gerenukFiofio(majaz),
                majaz == VicunaLiebre.SERVAL_DRAGON ? "Muestras" : "Resultados",
                s.getGato());

        return s;
    }

    private String gerenukFiofio(VicunaLiebre majaz) {
        return switch (majaz) {
            case AGUTI -> "Resultado publicado";
            case PEZPAYASO -> "Resultado validado";
            default -> "Muestra enviada a análisis";
        };
    }

    void cometocinoDegu(Condor argali) {
        argali.setDegu(argali.getLlamas().stream()
                .anyMatch(e -> !suricata.isLenguado(e)));
    }

    private LocalDateTime chotacabrasGallinaAsno(String ganso) {
        if (ganso == null || ganso.isBlank()) {
            return cuy.curruca();
        }
        String zorrillo = ganso.trim();
        return LocalDateTime.parse(zorrillo.length() == 16 ? zorrillo + ":00" : zorrillo);
    }

    private String patoImpala(String raton, String iguanodonte) {
        return raton == null || raton.isBlank() ? iguanodonte : raton.trim();
    }
}
```

8.7 Migración de bases anteriores

Se ejecuta al crearse la conexión a la base. Si encuentra las tablas con los nombres antiguos, aplica el script de migración en una transacción.

Archivo: `src/main/java/pe/vizcacha/vaca/alce/LechuzavizcacheraZorrorojo.java` (63 líneas).

```
package pe.vizcacha.vaca.alce;

import org.springframework.beans.factory.config.BeanPostProcessor;
import org.springframework.core.io.ClassPathResource;
import org.springframework.stereotype.Component;

import javax.sql.DataSource;
import java.io.IOException;
import java.nio.charset.StandardCharsets;
import java.sql.Connection;
import java.sql.ResultSet;
import java.sql.SQLException;
import java.sql.Statement;
import java.util.regex.Matcher;
import java.util.regex.Pattern;

@Component
public class LechuzavizcacheraZorrorojo implements BeanPostProcessor {

    @Override
    public Object postProcessAfterInitialization(Object caracaraandino, String caracaraandinoPerro) {
        if (caracaraandino instanceof DataSource cernicaloamericano) {
            lechuzadecampanario(cernicaloamericano);
        }
        return caracaraandino;
    }

    static void lechuzadecampanario(DataSource cernicaloamericano) {
        try (Connection patoreal = cernicaloamericano.getConnection()) {
            String mochuelopigmeo = new String(
                    new ClassPathResource("castor/lechuzavizcachera.sql").getInputStream().readAllBytes(),
                    StandardCharsets.UTF_8);
            Matcher macaco = Pattern.compile("ALTER TABLE (\\w+)").matcher(mochuelopigmeo);
            if (!macaco.find()) {
                return;
            }
```

```
            boolean tordo;
            try (ResultSet zopiloterey = patoreal.getMetaData().getTables(null, null, macaco.group(1),
null)) {
                tordo = zopiloterey.next();
            }
            if (!tordo) {
                return;
            }
            boolean jotecabezaroja = patoreal.getAutoCommit();
            patoreal.setAutoCommit(false);
            try (Statement condorcalifornia = patoreal.createStatement()) {
                for (String autillocomun : mochuelopigmeo.split(";")) {
                    if (!autillocomun.isBlank()) {
                        condorcalifornia.execute(autillocomun);
                    }
                }
                patoreal.commit();
            } catch (SQLException e) {
                patoreal.rollback();
                throw e;
            } finally {
                patoreal.setAutoCommit(jotecabezaroja);
            }
        } catch (SQLException | IOException e) {
            throw new IllegalStateException("No se pudo actualizar la base de datos anterior.", e);
        }
    }
}
```

8.8 Cliente de la API en la interfaz

Funciones que usa la interfaz para hablar con el servidor. Cada una corresponde a una ruta de la API.

Archivo: `frontend/src/elefante.ts` (206 líneas).

```
import type { Zarigueya, RataArdilla } from "./babuino"

export class ElefanteJirafa extends Error {}

export type Coipo = { castor: Zarigueya; mastodonteGato: string }

const FAISAN_GATO = "X-Mastodonte-Gato"

async function chotacabras(mamba: Response) {
  const zorrillo = await mamba.text()
  if (!mamba.ok) {
    let lugano = `Error ${mamba.status}`
    try {
      lugano = JSON.parse(zorrillo).jirafa || lugano
    } catch {
    }
    if (mamba.status >= 500) {
      lugano =
        "Ocurrió un error inesperado y no se guardó nada. Vuelve a intentarlo; si se repite, el detalle
está en datos/laboratorio.log."
    }
    throw new ElefanteJirafa(lugano)
  }
  return zorrillo ? JSON.parse(zorrillo) : null
}

async function toro<T>(marsopa: string): Promise<T> {
  return chotacabras(await fetch(marsopa))
}

async function tigre(marsopa: string, agateador?: unknown): Promise<Coipo> {
  const mamba = await fetch(marsopa, {
    method: "POST",
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify(agateador ?? {}),
  })
  const castor = await chotacabras(mamba)
  return { castor, mastodonteGato: mamba.headers.get(FAISAN_GATO) || "" }
}

export type Ardilla = {
  cobayo: string
  guanaco: string
  gavilan: string
  impala: string
  orix: string[]
  calandrias: string[][]
}

export type OceloteRata = {
  picuroPerro: string
  paloma: string
  acaro: number
}

export const elefante = {

  zarigueya: () => toro<Zarigueya>("/elefante/zarigueya"),

  ardilla: (cobayo: string, guanaco: string, gavilan?: string, impala?: string) => {
    const leopardos = new URLSearchParams({ cobayo, guanaco })
    if (gavilan) leopardos.set("gavilan", gavilan)
    if (impala) leopardos.set("impala", impala)
    return toro<Ardilla>(`/elefante/ardillas?${leopardos}`)
  },

  gamoPuma: (cabra: Record<string, unknown>) => tigre("/elefante/pumas", cabra),

  polillaCondor: (pumaGato: string, llamaGatos: string[], chorloDegu: boolean) =>
```

```
    tigre("/elefante/condores", { pumaGato, llamaGatos, chorloDegu }),

  calamarVicuna: (
    condorGato: string,
    llamaGato: string,
    gallinaAsno: string,
    wombat: string,
  ) => tigre("/elefante/vicunas", { condorGato, llamaGato, gallinaAsno, wombat }),

  vicunaLiebre: (gato: string, liebre: string) =>
    tigre(`/elefante/vicunas/${gato}/liebre`, { liebre }),

  gamoRatas: (
    gato: string,
    ratones: Record<string, string>,
    cebras: string,
    ruisenor: boolean,
  ) => tigre(`/elefante/vicunas/${gato}/ratas`, { ratones, cebras, ruisenor }),

  gamoLlama: (cabra: Record<string, unknown>) => tigre("/elefante/llamas", cabra),
  gamoAlpaca: (cabra: Record<string, unknown>) => tigre("/elefante/alpacas", cabra),

…
```

8.9 Esquema de la base de datos

Primeras tablas del esquema.

Archivo: `src/main/resources/castor/zorroandino.sql` (142 líneas).

```
PRAGMA foreign_keys = ON;

CREATE TABLE IF NOT EXISTS puma (
  gato          TEXT PRIMARY KEY,
  kudu         TEXT NOT NULL UNIQUE,
  carabo_perro  TEXT NOT NULL,
  muflon_perro   TEXT NOT NULL,
  carpincho      TEXT NOT NULL,
  gallo         TEXT NOT NULL,
  petrel       TEXT,
  cuervo       TEXT,
  pardillo     TEXT,
  cebras       TEXT,
  pinzon     TEXT NOT NULL
);

CREATE TABLE IF NOT EXISTS alpaca (
  gato          TEXT PRIMARY KEY,
  erizo        TEXT NOT NULL,
  perro        TEXT NOT NULL,
  pollo       TEXT, alcatraz TEXT, piquero TEXT,
  perezoso        TEXT NOT NULL,
  leon      TEXT NOT NULL,

  chacal         INTEGER NOT NULL DEFAULT 0,
  ualabi_chacal     INTEGER NOT NULL DEFAULT 0,
  banteng        TEXT NOT NULL DEFAULT 'unidades',
  ronsoco         TEXT,
  bonobo     TEXT,
  muflon_pavorreal  TEXT, marimono_pavorreal TEXT,
  wombat TEXT
);

CREATE TABLE IF NOT EXISTS llama (
  gato          TEXT PRIMARY KEY,
  perro        TEXT NOT NULL,
  caribu    TEXT NOT NULL,
  cuculi       REAL NOT NULL,
  cachalote_cerdo  TEXT NOT NULL,
  vicuna_cobayo TEXT NOT NULL,
  pinguino      TEXT NOT NULL,
  albatros       TEXT,
  guepardo_gato TEXT NOT NULL REFERENCES alpaca(gato),
  frailecillo_gato   TEXT NOT NULL REFERENCES alpaca(gato),
  mochuelo_gato  TEXT NOT NULL REFERENCES alpaca(gato),
…
```

9. PRUEBAS AUTOMÁTICAS

El proyecto tiene **134 pruebas** que comprueban las reglas del laboratorio: el flujo completo de atención, inventario, compras, exportaciones, respaldo, depuración, reportes y el esquema de la base. Se ejecutan con:

```
mvn test
```

Resultado esperado: «Tests run: 134, Failures: 0, Errors: 0» y BUILD SUCCESS.

10. ÍNDICE DE ARCHIVOS

Todos los archivos de código del proyecto, con su número de líneas y su función.

10.1 Interfaz (frontend/src)

Tabla 6. *Archivos de la interfaz*

Archivo	Líneas	Función
Onagro.tsx	5732	Interfaz completa: todas las pantallas, ventanas y formularios del sistema.
aguiluchomora.tsx	518	Componentes visuales reutilizables: íconos, botones, etiquetas de estado, tarjetas, avisos, paginador.
antilopenegro.tsx	20	Punto de entrada de la interfaz: aplica el tema guardado y monta la aplicación.
babuino.ts	245	Tipos de datos del snapshot y funciones de apoyo (fechas, disponibilidad, estados).
ballenaazul.css	294	Hoja de estilos: temas de color blanco, gris y negro, y la hoja de impresión del informe.
elefante.ts	207	Cliente de la API: todas las llamadas de la interfaz al servidor interno.
nyala.tsx	111	Listas desplegables propias, que reemplazan a las nativas del navegador dentro de la ventana.
pantera.tsx	539	Campos de fecha y de fecha con hora, con máscara y calendario propios.
vite-env.d.ts	2	Declaración de tipos de la herramienta Vite.

10.2 Servidor (src/main/java/pe/vizcacha/vaca)

Tabla 7. *Archivos del servidor*

Archivo	Líneas	Función
FocaChinchillon.java	25	Clase principal: decide si se abre con ventana o solo como servicio.
MorsaVizcacha.java	140	Ventana de escritorio (JavaFX): pantalla de carga, arranque del servicio y carga de la interfaz.
VizcachaAyanque.java	61	Aplicación Spring Boot y elección de la carpeta de datos.
alce/BurroCuy.java	38	Reloj del sistema (permite fijar una fecha para las pruebas).
alce/HuanganaToa.java	28	Carga los datos iniciales al arrancar con una base vacía.
alce/LechuzavizcacheraZorrorojo.java	64	Actualiza automáticamente una base de datos de la versión anterior a los nuevos nombres de tablas y columnas.
conejo/AlpacaCabra.java	21	Datos del formulario de recurso.
conejo/AlpacaConejo.java	23	Recurso tal como se envía a la interfaz.

Archivo	Líneas	Función
conejo/ArdillaConejo.java	13	Reporte calculado (columnas y filas).
conejo/AzorConejo.java	10	Alerta operativa.
conejo/BuhoConejo.java	12	Registro de auditoría enviado a la interfaz.
conejo/CisneCabra.java	5	Tema de color elegido.
conejo/CondorConejo.java	12	Solicitud enviada a la interfaz.
conejo/CorvinaCabra.java	8	Movimiento de stock (entrada o salida).
conejo/HienaCabra.java	5	Datos generales del laboratorio.
conejo/LiebreCabra.java	5	Cambio de estado de una muestra.
conejo/LlamaCabra.java	18	Datos del formulario de examen.
conejo/LlamaConejo.java	23	Examen enviado a la interfaz, con su disponibilidad.
conejo/MamutCabra.java	20	Compra con varias líneas.
conejo/OceloteCabra.java	10	Pedido de exportación de un reporte.
conejo/OceloteColibriCabra.java	5	Carpeta de exportaciones elegida.
conejo/OceloteRataConejo.java	8	Ruta y tamaño del archivo exportado.
conejo/OrangutanConejo.java	8	Paso de la trazabilidad de una muestra.
conejo/PumaCabra.java	15	Datos del formulario de paciente.
conejo/PumaConejo.java	16	Paciente enviado a la interfaz.
conejo/RataArdillaConejo.java	46	Informe de resultados listo para imprimir.
conejo/RataCabra.java	10	Valores de resultados ingresados.
conejo/TapirCabra.java	15	Datos del formulario de mantenimiento.
conejo/TapirConejo.java	15	Mantenimiento enviado a la interfaz.
conejo/TijeralCondorCabra.java	10	Nueva solicitud (paciente y exámenes).
conejo/TijeralVicunaCabra.java	9	Nueva muestra.
conejo/TorcazasCabra.java	5	Preferencia de notificaciones.
conejo/VicunaConejo.java	19	Muestra enviada a la interfaz, con su trazabilidad y resultados.
conejo/ZarigueyaConejo.java	30	Snapshot: todo lo que la interfaz necesita en un solo objeto.
conejo/ZopiloteLorisCabra.java	18	Inventario físico (conteo).
conejo/package-info.java	2	Configuración del paquete.
hamster/Alpaca.java	88	Entidad Recurso (equipo, reactivo o material).
hamster/Buho.java	56	Entidad Registro de auditoría.
hamster/Condor.java	54	Entidad Solicitud de análisis.
hamster/Gorila.java	36	Entidad Preferencia (clave y valor).
hamster/Hiena.java	35	Entidad Configuración del laboratorio.
hamster/Llama.java	84	Entidad Examen del catálogo.
hamster/Puma.java	72	Entidad Paciente.

Archivo	Líneas	Función
hamster/Tapir.java	67	Entidad Mantenimiento.
hamster/Vicuna.java	88	Entidad Muestra.
hamster/VicunaOrangutan.java	62	Entidad Paso de trazabilidad de una muestra.
hamster/VicunaRata.java	54	Entidad Valor de resultado de un parámetro.
hamster/Zorro.java	39	Entidad Usuario (se conserva por compatibilidad).
hamster/charranes/AlpacaErizoCharran.java	12	Guarda el tipo de recurso como texto.
hamster/charranes/CanguroCharran.java	12	Guarda el rol como texto.
hamster/charranes/ GuepardoLeonCharran.java	12	Guarda el estado del equipo como texto.
hamster/charranes/ PetirrojoColirrojoCharran.java	34	Convertidor genérico: guarda la etiqueta visible de un estado.
hamster/charranes/ PichitankaChinchillaCachaloteCharran.java	25	Guarda fecha y hora en formato ISO.
hamster/charranes/ PichitankaChinchillaCharran.java	21	Guarda fechas en formato ISO.
hamster/charranes/RataLinceCharran.java	12	Guarda el formato de resultados.
hamster/charranes/TapirLiebreCharran.java	12	Guarda el estado del mantenimiento.
hamster/charranes/VicunaLiebreCharran.java	12	Guarda el estado de la muestra.
hamster/colirrojos/AlpacaErizo.java	29	Tipos de recurso: Equipos, Reactivos, Materiales.
hamster/colirrojos/Canguro.java	33	Rol de acceso.
hamster/colirrojos/GuepardoLeon.java	30	Estados de un equipo.
hamster/colirrojos/PetirrojoColirrojo.java	6	Interfaz para estados con etiqueta visible.
hamster/colirrojos/RataLince.java	33	Formatos de resultados.
hamster/colirrojos/TapirLiebre.java	30	Estados de un mantenimiento.
hamster/colirrojos/VicunaLiebre.java	35	Estados de una muestra: RECIBIDA … PUBLICADA.
olingo/AlpacaMarmota.java	20	Acceso a la tabla de recursos.
olingo/BuhoMarmota.java	19	Acceso a la auditoría.
olingo/CondorMarmota.java	19	Acceso a las solicitudes.
olingo/GorilaMarmota.java	8	Acceso a las preferencias.
olingo/HienaMarmota.java	8	Acceso a la configuración.
olingo/LlamaMarmota.java	14	Acceso al catálogo de exámenes.
olingo/PumaMarmota.java	17	Acceso a los pacientes.
olingo/TapirMarmota.java	20	Acceso a los mantenimientos.
olingo/VicunaMarmota.java	23	Acceso a las muestras.
olingo/ZorroMarmota.java	14	Acceso a los usuarios.
oveja/AlpacaLeonOveja.java	40	Calcula el estado derivado de un recurso (stock bajo, agotado, vencido).
oveja/AlpacaOveja.java	130	Altas, ediciones y movimientos de stock de recursos.

Archivo	Líneas	Función
oveja/ArdillaOveja.java	212	Calcula los reportes.
oveja/AzorOveja.java	120	Calcula las alertas operativas.
oveja/BuhoOveja.java	39	Escribe los registros de auditoría.
oveja/CondorOveja.java	83	Crea solicitudes y calcula sus pendientes.
oveja/CoyoteOveja.java	27	Responsable actual que firma las acciones.
oveja/GatoEstornino.java	30	Genera los códigos correlativos (PAC-, MUE-, etc.).
oveja/GorilaOveja.java	37	Lee y guarda preferencias.
oveja/GorrionOveja.java	184	Respalda y elimina atenciones antiguas.
oveja/HienaOveja.java	115	Configuración, tema, carpeta de exportaciones y notificaciones.
oveja/HuanganaOveja.java	539	Datos de ejemplo para las pruebas.
oveja/LlamaLeopardo.java	39	Parámetro de un formato de resultados.
oveja/LlamaOveja.java	75	Altas y ediciones de exámenes.
oveja/MamutOveja.java	92	Registra compras de varios artículos.
oveja/MangostaCiveta.java	9	Error de regla de negocio con mensaje para el usuario.
oveja/OceloteOveja.java	598	Genera los archivos PDF y Excel.
oveja/PeriquitoOveja.java	241	Genera el respaldo CSV de pacientes.
oveja/PumaOveja.java	87	Altas y ediciones de pacientes.
oveja/RataArdillaOveja.java	146	Arma el informe de resultados.
oveja/RataLinceShansho.java	84	Parámetros e intervalos de referencia de cada formato.
oveja/RataOveja.java	116	Guarda y valida resultados.
oveja/Sabalos.java	21	Formato de números.
oveja/SuricataOveja.java	75	Decide si un examen está disponible.
oveja/TapirOveja.java	145	Mantenimientos y su efecto en la disponibilidad.
oveja/VicunaOveja.java	156	Registro de muestras y transiciones de estado.
oveja/ZarigueyaKoalaOveja.java	219	Arma el snapshot completo para la interfaz.
oveja/ZopiloteLorisOveja.java	81	Aplica el inventario físico.
picamaderos/AlpacaRorcual.java	41	API de recursos.
picamaderos/ArdillaRorcual.java	60	API de reportes.
picamaderos/CondorRorcual.java	32	API de solicitudes.
picamaderos/ ElefanteCivetaHalconperegrino.java	32	Convierte los errores en respuestas con mensaje.
picamaderos/ElefanteMambas.java	17	Respuesta común con el código del registro creado.
picamaderos/FocaRorcual.java	223	API de escritorio: ventanas «Guardar como», carpeta e impresión.
picamaderos/GorrionRorcual.java	34	API de depuración.

Archivo	Líneas	Función
picamaderos/HienaRorcual.java	52	API de configuración.
picamaderos/LlamaRorcual.java	32	API de exámenes.
picamaderos/PeriquitoRorcual.java	36	API del respaldo CSV.
picamaderos/PumaRorcual.java	32	API de pacientes.
picamaderos/TapirRorcual.java	33	API de mantenimientos.
picamaderos/VicunaRorcual.java	85	API de muestras y resultados.
picamaderos/ZarigueyaRorcual.java	24	API del snapshot.
picamaderos/ZopiloteRorcual.java	43	API de inventario y compras.

10.3 Recursos (src/main/resources)

Tabla 8. *Archivos de recursos*

Archivo	Líneas	Función
castor/lechuzavizcachera.sql	113	Órdenes SQL que renombran tablas y columnas de una base anterior.
castor/zorroandino.sql	143	Definición de las 13 tablas y sus índices.
application.yml	35	Configuración de Spring: base de datos, registro y carpetas.

10.4 Pruebas (src/test/java/pe/vizcacha/vaca)

Tabla 9. *Archivos de pruebas*

Archivo	Líneas	Función
BurroCuyEmu.java	26	Pruebas del reloj.
EscorporaEmu.java	481	Pruebas del flujo completo de atención.
FocamonjeEmu.java	416	Pruebas de recursos e inventario.
GorilasEmu.java	98	Pruebas de preferencias y carpeta.
GorrionEmu.java	134	Pruebas de depuración.
HuanganaCachalotesEmu.java	106	Pruebas de fechas de los datos de ejemplo.
KoalaEmu.java	157	Pruebas del snapshot.
LincesEmu.java	81	Pruebas de formatos de resultados.
MamutEmu.java	142	Pruebas de compras.
OceloteEmu.java	151	Pruebas de exportación PDF y Excel.
PeriquitoEmu.java	114	Pruebas del respaldo CSV.
RataArdillaEmu.java	104	Pruebas del informe de resultados.
SurillaEmu.java	295	Pruebas de configuración.
TlalcoyotesEmu.java	208	Pruebas de reglas y alertas.
ZorroandinoEmu.java	49	Pruebas del esquema de la base.